JavaScript / HOW TO TRACK ERRORS IN A WEB BROWSER

In a web browser, you can use ***developer tools*** to track errors in JavaScript code. The unobtrusive JavaScript scripts never generate error messages on the browser window, even when they fail. If features cannot be presented properly, they silently disappear. This makes error tracking harder.

NB! When you ask teacher's help in JavaScript error tracking, please (a) make sure that ***your HTML code is valid*** and (b) be prepared to show

1. What **run-time error message(s) are shown in the *Console window*** when you run the program
2. What **error message(s) etc. the code editor shows** (if any)
3. How you have **debugged your JavaScript code** and **explain where the code fails**.

1. Web browsers log run-time errors to Web **Console Window.**

   To see the Console Window, press **F12** to open *Developer tools* and then select the **Console** Window.

2. The Web Browser's **Debugger** tool lets you to execute the code line by line to see where the code fails and what are the current values of variables.

   Debugging in Chrome: https://developers.google.com/web/tools/chrome-devtools

   Debugging in Firefox: https://developer.mozilla.org/en-US/docs/Tools/Debugger

HOW TO DEBUG	
Open the debugger	Press **F12** and select the **Debugger** window
Select the source file	Select the JavaScript file in the **Sources** pane. - If there is no JavaScript file name, then refresh the web page.
Set a one or more breakpoints	In the **JavaScript source pane**, click on the line number for the line you want to break at.
Execute JavaScript	For example, click on a button on your web page or refresh the page
Step through the code	When your code stops at a breakpoint, you can step through it using the following **four buttons on the *Developer toolbar***: Chrome FireFox • **Resume** (F8): continue running to the next breakpoint • **Step over** (F10): advance to the next line in the same function. • **Step into** (**F11**): advance to the next line in the function, unless on a function call, in which case enter the function being called • **Step out** (Shift-F11): run to the end of the current function
Examine variables ▼ Scope ▼ Local age: 25 ageText: "25" outputText: undefined * Examine expressions * Change values of variables manually	When the code has stopped at a breakpoint, you can examine its *local* and *global* variables in the **Scope pane** (Chrome) or in the **Variables pane** (Firefox). • In Chrome, you can also hover the mouse pointer on a variable name in the code to see its current value. In the **Watch pane** (Chrome) or in the **Variables pane** (Firefox), you can **add a watch** for a *variable* or *any expression* to make its value visible all the time. There you can also change values of variables manually.
Disable a breakpoint	To disable a breakpoint, uncheck the checkbox next to the breakpoint's entry in the **Sources pane** or click on the breakpoint on the line number.